

Day 26: エラー対策・デバッグを学ぼう

今日のゴール

よくあるエラーのを見つけ方と直し方を学びます。ブラウザの開発者ツールを使ったデバッグ方法を習得します。

【スクリーンショット: Chrome DevToolsのConsoleとNetwork】

なぜこれを学ぶのか?

開発では必ずエラーに遭遇します。**デバッグは探偵のようなもの**。事件(エラー)が起きたとき、証拠(ログ)を集めて、犯人(バグ)を特定します。デバッグスキルがあれば、どんなエラーでも怖くありません。

学習ステップ一覧

ステップ	学習内容	所要時間
Step 1	Chrome DevToolsの使い方	15分
Step 2	よくあるエラーと対処法	20分
Step 3	ネットワークエラーのデバッグ	15分
Step 4	エラーハンドリングの追加	15分

合計時間: 約65分

Step 1: Chrome DevToolsの使い方（15分）

開発者ツールを開く:

Windows/Linux: F12 または Ctrl + Shift + I
Mac: Cmd + Option + I

主要なタブ:

- **Console:** エラーメッセージ、console.log()の出力
- **Network:** APIリクエストの確認
- **Application:** LocalStorage、Cookie、Session
- **Sources:** ブレークポイントでデバッグ

Console の使い方:

```
// filepath: src/app/dashboard/page.tsx
export default function DashboardPage() {
  const { data: stats } = api.report.getStats.useQuery();

  console.log('統計データ:', stats);
  console.error('これはエラーです');
  console.warn('これは警告です');
  console.table(stats); // テーブル形式で表示

  return (
    // UI
  );
}
```

確認ポイント: Consoleにログが表示される

Step 2: よくあるエラーと対処法 (20分)

エラー1: Cannot read property 'xxx' of undefined

```
// × エラーが出る例
const taskTitle = task.title; // task が undefined

// 修正方法
const taskTitle = task?.title; // オプショナルチェーン
const taskTitle = task?.title ?? 'デフォルト値'; // Nullish
```

エラー2: Maximum update depth exceeded

```
// × 無限ループの例
useEffect(() => {
  setCount(count + 1); // 依存配列に count がある
}, [count]);

// 修正方法
useEffect(() => {
  setCount(prev => prev + 1); // 関数形式を使う
}, []); // 依存配列を正しく設定
```

エラー3: Hydration failed

```
// × SSRとクライアントで異なる
export default function Component() {
  return <div>{new Date().toString()}</div>;
}

// 修正方法
'use client';
import { useState, useEffect } from 'react';

export default function Component() {
  const [mounted, setMounted] = useState(false);

  useEffect(() => {
    setMounted(true);
  }, []);

  if (!mounted) return null;

  return <div>{new Date().toString()}</div>;
}
```

確認ポイント: エラーの原因と対処法を理解する

Step 3: ネットワークエラーのデバッグ (15分)

Networkタブの使い方:

1. DevToolsを開く
2. Networkタブをクリック
3. ページをリロード
4. 失敗したリクエストは赤く表示される

tRPCエラーの確認:

```
// filepath: src/app/dashboard/page.tsx
export default function DashboardPage() {
  const { data, error, isLoading } = api.report.getStats.

  if (error) {
    console.error('APIエラー:', error);
    return (
      <Box sx={{ p: 3 }}>
        <Alert severity="error">
          データの取得に失敗しました: {error.message}
        </Alert>
      </Box>
    );
  }

  // 通常の処理
}
```

エラーの種類:

- **400 Bad Request:** リクエストのパラメータが間違っている
- **401 Unauthorized:** ログインが必要
- **403 Forbidden:** 権限がない
- **404 Not Found:** URLが間違っている
- **500 Internal Server Error:** サーバー側のエラー

確認ポイント: Networkタブでエラーを確認できる

Step 4: エラーハンドリングの追加 (15分)

グローバルエラーハンドリング:

```

// filepath: src/app/error.tsx
'use client';

import { Box, Typography, Button } from '@mui/material';
import { useEffect } from 'react';

export default function Error({
  error,
  reset,
}: {
  error: Error;
  reset: () => void;
}) {
  useEffect(() => {
    console.error('エラー発生:', error);
  }, [error]);

  return (
    <Box sx={{ p: 3, textAlign: 'center' }}>
      <Typography variant="h4" sx={{ mb: 2 }}>
        エラーが発生しました
      </Typography>
      <Typography variant="body1" sx={{ mb: 3 }}>
        {error.message}
      </Typography>
      <Button variant="contained" onClick={reset}>
        再試行
      </Button>
    </Box>
  );
}

```

Try-Catch でエラーを捕捉:

```
// filepath: src/app/profile/page.tsx
const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();

  try {
    await updateProfileMutation.mutateAsync({ name, email });
    alert('更新しました');
  } catch (error) {
    console.error('更新エラー:', error);
    alert('更新に失敗しました');
  }
};
```

確認ポイント: エラーが適切に処理される

学んだこと

- **Chrome DevTools**: Consoleでログ確認、Networkでリクエスト確認
- **オプションナルチェーン (?.)**: undefinedエラーを防ぐ
- **useEffect の依存配列**: 無限ループを防ぐ
- **Hydration エラー**: SSRとクライアントの不一致を解決
- **HTTPステータスコード**: エラーの種類を理解
- **error.tsx**: Next.jsのグローバルエラーハンドリング

今日のまとめ

- ☐ Chrome DevToolsの使い方を学んだ
- ☐ よくあるエラーと対処法を理解した
- ☐ ネットワークエラーのデバッグ方法を習得した
- ☐ エラーハンドリングを追加できた

⚠ デバッグのコツ

やること	理由
エラーメッセージを読む	エラーの原因が書いてある
<code>console.log()</code> を追加	変数の中身を確認
Networkタブを確認	APIエラーの詳細を見る
1つずつ変更して確認	何が原因か特定する

次回予告

Day 27では、セキュリティ対策について学びます。